BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 1

Roll no: 37 PRN No: 12412175 Name of Student: Rugved Raut

Subject Name : Deep Learning

Problem Statement :Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

Alogorithm :

1. Start the program.
2. Import the required libraries: TensorFlow, Keras, NumPy, and Matplotlib.
3. Load the MNIST dataset using mnist.load_data().
4. Display the shape of the training and testing datasets.
5. Check the data type and pixel value range of the images.
6. Display sample images with their corresponding labels.
7. Convert the image data type from uint8 to float32.
8. Normalize the pixel values by dividing each pixel by 255.0.
9. Reshape the images from (28, 28) to (28, 28, 1).
10. Display a normalized sample image.
11. Plot the class distribution of the training dataset.
12. Plot the histogram of normalized pixel intensities.
13. Display multiple sample images for visualization.
14. Verify the train-test split and dataset dimensions.
15. Display the final dataset summary.
16. Stop the program.

Code :

```
import tensorflow as tf
import matplotlib.pyplot as plt
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.mnist.load_data()
train_images = train_images /255.0
test_images = test_images / 255.0
print(train_images.shape)
```

```
print(test_images.shape)
print(train_labels)

plt.imshow(train_images[1], cmap='gray')
plt.show()
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))     #Rectified Linear Unit        #Dense layers are used to
learn complex patterns in data.
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))

my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

my_model.fit(train_images, train_labels, epochs=3)
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)
my_model.save('my_mnist_model')
```

Output :

```
Training Shape : (60000, 28, 28, 1)
Testing Shape  : (10000, 28, 28, 1)
```



